RExercise#6_Porras

Renz Benedict Porras

2026-10-22

Part A — Creating Data Frames

```
# TODO: Create three vectors of equal length (5 students):
# name (character): "Ana", "Ben", "Cid", "Dex", "Eli"
# age (numeric): 20, 21, 19, 22, 20
# passed (logical): TRUE, TRUE, FALSE, TRUE, FALSE

name <- c("Ana", "Ben", "Cid", "Dex", "Eli")
age <- c(20, 21, 19, 22, 20)
passed <- c(TRUE, TRUE, FALSE, TRUE, FALSE)
```

```
# TODO: Combine name, age, and passed into a data frame called students
# using data.frame(name = name, age = age, passed = passed)
# Print students

students <- data.frame(name = name, age = age, passed = passed)

students
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 3  Cid  19  FALSE
## 4  Dex  22   TRUE
## 5  Eli  20  FALSE
```

```
# TODO: Check the type of the "name" column using typeof(students$name)
# and class(students$name)

typeof(students$name)
```

```
## [1] "character"
```

```
class(students$name)
```

```
## [1] "character"
```

```
# TODO: Now create a SECOND data frame called courses directly, without
# separate vectors first, using data.frame() with these columns:
# course_code = c("IS101", "IS102", "IS103")
# units = c(3, 3, 2)
# instructor = c("Dr. P. Reyes", "Dr. R. Santos", "Dr. S. Cruz")

courses <- data.frame(
  course_code = c("IS101", "IS102", "IS103"),
```

```
  units = c(3, 3, 2),
  instructor = c("Dr. P. Reyes", "Dr. R. Santos", "Dr. S. Cruz")
)

courses
```

```
##   course_code units    instructor
## 1       IS101     3  Dr. P. Reyes
## 2       IS102     3 Dr. R. Santos
## 3       IS103     2   Dr. S. Cruz
```

Question #1:

```
# You created students two different ways in concept: first from separate vectors, then courses directly
# Both approaches produce the same kind of object.
# In your own words, explain why data.frame() requires every column argument to have the same length, a

# A data frame is a rectangular table where every row represents one complete record. All columns must
# If the vector lengths do not match, R cannot form a rectangular grid and throws an error such as:
# "Error in data.frame(...) : arguments imply differing number of rows"
```

Question #2:

```
# A data frame is often described as "a list of vectors of equal length."
# Based on what you know about lists from a previous lab, explain why this description makes sense - sp

# Lists are designed to hold elements of different data types (such as text, numbers, or booleans).
# Since a data frame is a list of equal-length vectors, each column can store a different data type (fo
# However, because each individual column is a single atomic vector, all values inside that specific co
```

Part B — Subsetting Data Frames

```
# TODO: Access the "age" column of students three different ways:
# students$age
# students[["age"]]
# students["age"] -- note: compare the output/class of this one to the others

students$age
```

```
## [1] 20 21 19 22 20
```

```
students[["age"]]
```

```
## [1] 20 21 19 22 20
```

```
students["age"]
```

```
##   age
## 1  20
## 2  21
## 3  19
## 4  22
## 5  20
```

```r
# TODO: Access the entire 2nd row of students using students[2, ]

students[2, ]
```

```
##   name age passed
## 2  Ben  21   TRUE
```

```r
# TODO: Access the value in row 3, column "name" using students[3, "name"]

students[3, "name"]
```

```
## [1] "Cid"
```

```r
# TODO: Return only the rows of students where passed is TRUE
# Hint: students[students$passed == TRUE, ]

students[students$passed == TRUE, ]
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 4  Dex  22   TRUE
```

```r
# TODO: Return only the rows of students where passed is TRUE AND
# age is greater than 19. Use the & operator to combine both conditions.

students[students$passed == TRUE & students$age > 19, ]
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 4  Dex  22   TRUE
```

Question #3:

```r
# students["age"] and students$age both seem to give you the age column, but class() reveals they are d
# Run class() on each and explain the difference, connecting it to what you learned about single vs. do

class(students["age"])
```

```
## [1] "data.frame"
```

```r
class(students$age)
```

```
## [1] "numeric"
```

```r
# Explanation:
# students["age"] keeps the 1-column data frame structure (just like single brackets [] on a list keep
# students$age extracts the raw values as a simple 1D numeric vector (just like double brackets [[]] or
```

Question #4:

```r
# In df-multiple-conditions, explain what the & operator is doing to the two logical vectors (students$
# What would change if you used | (OR) instead of & (AND)?
```

```
# Explanation:
# The & (AND) operator compares both logical vectors element-by-element and returns TRUE only when BOTH
# If you used | (OR) instead, R would return TRUE if EITHER condition is met. This would include studen
```

Part C — Data Frame Properties

```
# TODO: Use str() on students to see the structure and types of each column

str(students)
```

```
## 'data.frame':    5 obs. of  3 variables:
##  $ name  : chr  "Ana" "Ben" "Cid" "Dex" ...
##  $ age   : num  20 21 19 22 20
##  $ passed: logi  TRUE TRUE FALSE TRUE FALSE
```

```
# TODO: Use dim(), nrow(), and ncol() on students to report its size

dim(students)
```

```
## [1] 5 3
```

```
nrow(students)
```

```
## [1] 5
```

```
ncol(students)
```

```
## [1] 3
```

```
# TODO: Use names() (or colnames()) to list the column names of students

names(students)
```

```
## [1] "name"   "age"    "passed"
```

```
# TODO: Use summary() on students to get summary statistics for each column

summary(students)
```

```
##         name        age         passed
##  Length   :5   Min.   :19.0   Mode :logical
##  N.unique :5   1st Qu.:20.0   FALSE:2
##  N.blank  :0   Median :20.0   TRUE :3
##  Min.nchar:3   Mean   :20.4
##  Max.nchar:3   3rd Qu.:21.0
##                Max.   :22.0
```

Question #5:

```
# Run summary() on students. Notice that the numeric column (age) and the logical column (passed) are s
# Describe what kind of summary R produces for each of the three data types, and explain why a "mean" w

summary(students)
```

```
##         name        age         passed
##  Length   :5   Min.   :19.0   Mode :logical
##  N.unique :5   1st Qu.:20.0   FALSE:2
```

```
##  N.blank  :0   Median :20.0   TRUE :3
##  Min.nchar:3   Mean   :20.4
##  Max.nchar:3   3rd Qu.:21.0
##                Max.   :22.0
```

```
# Explanation:
# 1. Numeric (age): R provides five-number statistics plus the average (Min, 1st Qu, Median, Mean, 3rd
# 2. Logical (passed): R counts the frequency of TRUE and FALSE entries.
# 3. Character (name): R displays general metadata (Length, Class, and Mode) instead of statistical val
# A mean does not make sense for the "name" column because text strings are qualitative labels, not num
```

Question #6:

```
# str() and summary() both describe a data frame, but they show different information.
# Based on what you observed, when would str() be more useful to you than summary(), and vice versa?

# Explanation:
# - str() is more useful when checking the structural setup of a dataset (e.g., confirming column data
# - summary() is more useful when analyzing data distributions, checking summary statistics (like min,
```

Part D — Data Frame Operations

```
# TODO: Add a new column called year_level to students with values:
# 2, 3, 1, 4, 2  (one per student, in the same order as the rows)

students$year_level <- c(2, 3, 1, 4, 2)

students
```

```
##   name age passed year_level
## 1  Ana  20   TRUE          2
## 2  Ben  21   TRUE          3
## 3  Cid  19  FALSE          1
## 4  Dex  22   TRUE          4
## 5  Eli  20  FALSE          2
```

```
# TODO: Remove the "passed" column from students by setting it to NULL
# students$passed <- NULL

students$passed <- NULL

students
```

```
##   name age year_level
## 1  Ana  20          2
## 2  Ben  21          3
## 3  Cid  19          1
## 4  Dex  22          4
## 5  Eli  20          2
```

```
# TODO: Sort students by age in ascending order using order()
# Hint: students[order(students$age), ]

students[order(students$age), ]
```

```
##   name age year_level
## 3  Cid  19          1
## 1  Ana  20          2
## 5  Eli  20          2
## 2  Ben  21          3
## 4  Dex  22          4
```

```r
# TODO: Sort students by age in DESCENDING order
# Hint: use order(students$age, decreasing = TRUE), or order(-students$age)

students[order(students$age, decreasing = TRUE), ]
```

```
##   name age year_level
## 4  Dex  22          4
## 2  Ben  21          3
## 1  Ana  20          2
## 5  Eli  20          2
## 3  Cid  19          1
```

Question #7:

```r
# After removing the passed column, could you still run the filter from df-logical-filter (students$pas
# Explain what happens and why keeping track of a data frame's current structure matters as you apply a

# Explanation:
# No, running students$passed == TRUE will not work as expected. Since the passed column was removed, s
# Keeping track of a data frame's structure matters because script operations are sequential-deleting o
```

Question #8:

```r
# order() does not sort the data frame directly - it returns a vector of row positions.
# Explain, step by step, how students[order(students$age), ] uses that position vector to reorder the w

# Step-by-step explanation:
# 1. order(students$age) evaluates the values in the age column and returns a vector of row index numbe
# 2. This position vector is placed into the row slot of the subsetting bracket: students[row_positions
# 3. R takes entire rows and rearranges them into the exact sequence matching those index numbers.
# 4. Leaving the column slot blank after the comma (,) instructs R to keep all columns included, ensuri
```

Part E — Critical Thinking Challenge

```r
grades <- data.frame(
  name = c("Ana", "Ben", "Cid", "Dex", "Eli"),
  midterm = c(85, 78, 92, 88, 70),
  final = c(90, 82, 95, 84, 75)
)

# TODO: Add a new column "average" to grades that holds the mean of
# midterm and final for each student. Hint: (grades$midterm + grades$final) / 2

grades$average <- (grades$midterm + grades$final) / 2
```

```
grades
```

```
##   name midterm final average
## 1  Ana      85    90    87.5
## 2  Ben      78    82    80.0
## 3  Cid      92    95    93.5
## 4  Dex      88    84    86.0
## 5  Eli      70    75    72.5
```

```
# TODO: Print only the rows of grades where remarks is "Failed",
# showing just the name and average columns.
# Hint: grades[grades$remarks == "Failed", c("name", "average")]

# Create the remarks column if not already defined (passing grade = 75)
grades$remarks <- ifelse(grades$average < 75, "Failed", "Passed")

# Filter for "Failed" students displaying only name and average
grades[grades$remarks == "Failed", c("name", "average")]
```

```
##   name average
## 5  Eli    72.5
```

Question #9:

```
# Suppose your instructor wants to know: "Which factor seems more responsible for a student's final rem
# Using only the tools from this lab (subsetting, new columns, sorting), describe - in plain language,

# 1. New Column: Create a score drop column to measure performance changes:
#    grades$score_drop <- grades$midterm - grades$final

# 2. Subsetting: Filter the dataset to isolate only the failing students:
#    failing_students <- grades[grades$remarks == "Failed", ]

# 3. Sorting & Inspection:
#    - Sort failing_students by midterm score in ascending order to see how many started with low score
#    - Sort failing_students by score_drop in descending order to see how many experienced a steep drop

# 4. Compare whether low midterm scores or high drop values account for the majority of failing cases.
```

CONCLUSION

```
# In 3-4 sentences, summarize what a data frame is, how it relates to vectors and lists from earlier la

# A data frame is a two-dimensional tabular grid in R where rows represent observations and columns rep
```